

Introduction to Python

Module 0 — Setting Up Python

Ways to run Python, editors & IDEs, and virtual environments

What Is Python, Really?

Python is a programming language — a way of writing instructions a computer can follow. To *run* those instructions, you need the **Python interpreter**: a program that reads your code and executes it.

Good to know

- Free & open source
- Readable, beginner-friendly
- Huge library ecosystem
- The standard for data science

One historical wrinkle

- **Python 2** vs **Python 3**
- Python 2 is **dead** (retired 2020)
- Always use **Python 3**
- We use Python 3.10+

You write .py files or notebook cells; the interpreter turns them into actions.

Three Ways to Run Python

1. In the browser

Google Colab

- Zero installation
- Free, runs in the cloud
- Perfect for learning

We start here.

2. On your computer

Local install

- Full control
- Works offline
- Needed for real projects

Covered in this module.

3. The terminal

Interactive shell

- Type `python`
- Quick experiments
- Run `.py` scripts

Handy to know.

*For this course, **Colab** is all you need. The rest of this module is for when you set up Python on your own machine.*

Installing Python Locally

Two common routes. Both work — pick one.

python.org (official)

- Download the installer for your OS
- Lightweight, “just Python”
- You add packages yourself with `pip`

Good if you like control.

Anaconda / Miniconda

- Python *plus* data-science tools bundled
- **Anaconda**: big, everything included
- **Miniconda**: minimal, add what you need

Popular in science; Miniconda is a nice middle ground.

On Windows, tick “Add Python to PATH” during install — it saves a lot of headaches later.

Where You Write Code: Editors & IDEs

An **IDE** (Integrated Development Environment) is a program for writing, running, and debugging code. A few popular ones:

VS Code

Free, lightweight, hugely popular. Great all-rounder — our recommendation

Jupyter
JupyterLab

/

Notebook interface (like Colab, but local). Ideal for data analysis

PyCharm

Powerful, full-featured IDE. More for large software projects

Spyder

MATLAB-like; comes with Anaconda. Popular with scientists

*There is no single “right” one. For general Python work we suggest **VS Code**; for data analysis, many people live in **notebooks**.*

Getting Started with VS Code

VS Code (Visual Studio Code) is a free editor from Microsoft. Setting it up for Python takes a few steps:

1. Download from `code.visualstudio.com`
2. Install the **Python extension** (from the Extensions panel)
3. Open a folder for your project
4. Create a `.py` file and start writing

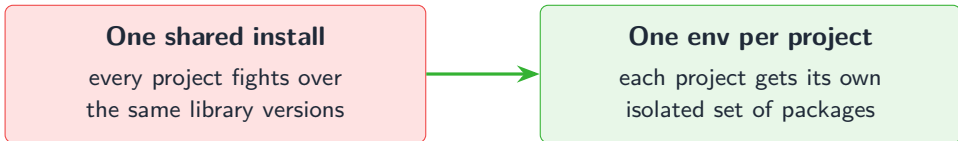
Why people love it

- Runs code with one click
- Highlights errors as you type
- Built-in terminal
- Handles notebooks *and* scripts
- Endless extensions

VS Code can also open `.ipynb` notebooks directly — so you get the best of both worlds.

Virtual Environments: The Why

Here is a problem you *will* hit. Project A needs version 1 of a library; project B needs version 2. Install them globally and they **clash**.



A **virtual environment** is an isolated Python setup for a single project. Its packages live in their own folder and don't interfere with anything else. **Use one per project** — it is the single best habit you can build early.

Virtual Environments: The How (venv)

Python's built-in tool is venv. In a terminal, inside your project folder:

```
# create an environment called ".venv"
python -m venv .venv

# activate it (Mac/Linux)
source .venv/bin/activate

# activate it (Windows)
.venv\Scripts\activate

# now install packages - they go inside .venv only
pip install pandas numpy
```

When activated, your prompt shows `(.venv)`. Type `deactivate` to leave. Everything you `pip install` while active stays **inside** this environment.

Conda & Sharing Your Setup

The conda alternative

If you installed Anaconda/Miniconda, conda does the same job:

```
conda create -n myenv python=3.11  
conda activate myenv
```

Handles some non-Python dependencies more smoothly.

Sharing: requirements.txt

List your project's packages so others can recreate it:

```
pip freeze > requirements.txt  
pip install -r requirements.txt
```

This is how reproducible projects travel between machines.

*venv + pip or conda — either is fine. The key idea is the same: **isolate, then record**.*

Getting Code from GitHub: Cloning

Course materials, shared projects, and most open-source code live on **GitHub**. To get a copy onto your own machine, you **clone** the repository.

The words

- **Git**: version-control software
- **GitHub**: a website that hosts Git repositories
- **Repository (“repo”)**: a project folder
- **Clone**: download a full copy

Why clone (vs. download)?

- You get the **full history**
- Easy to **pull updates** later
- Can contribute changes back
- The standard way to work with code

You only need the repository’s URL — found on its GitHub page under the green “Code” button.

Four Ways to Clone a Repository

1. **The terminal** (the classic way) — `cd` into where you want it, then:

```
git clone https://github.com/username/repo-name.git
```

2. **GitHub Desktop** — a friendly app, no terminal. *File* → *Clone Repository*, paste the URL, pick a folder. Great for beginners.
3. **VS Code** — built in. Open the Command Palette (`Ctrl/Cmd+Shift+P`), type “*Git: Clone*”, paste the URL.
4. **Download ZIP** (not really cloning) — the green **Code** button → *Download ZIP*. Quick, but you lose history and can’t easily pull updates.

To update a cloned repo later (get the latest changes), run `git pull` inside its folder — or click “Pull” in GitHub Desktop / VS Code.

Module 0: Takeaways

- ✓ Python is a language; the **interpreter** runs it — always use **Python 3**
- ✓ Run it in **Colab** (easiest), a **local install**, or the **terminal**
- ✓ Write code in an **IDE** — **VS Code** is a great default; notebooks suit data work
- ✓ Use a **virtual environment per project** (venv or conda) to avoid version clashes
- ✓ Record your packages in `requirements.txt` for reproducibility
- ✓ **Clone** a GitHub repo (terminal, GitHub Desktop, VS Code) to get code onto your machine
- ✓ **For this course, Colab is enough** — come back here when you go local

Next: Module 1 — writing your first actual Python code.